Golang Backend Developer Challenge

Objective

The purpose of this challenge is to evaluate your **backend development skills using Golang**. You will build an API for managing a logistics company that ships goods via vessels.

Technical Information

- Use **Golang (1.21+)**.
- Follow a **clean, modular architecture** (handlers/controllers, services, repositories, models).
- Apply **Go best practices** (interfaces, error wrapping, idiomatic style).
- Use **structured logging** (Zap, Zerolog, or stdlib log with context).
- Handle errors gracefully with meaningful messages and proper HTTP codes.
- Use **Go modules** for dependency management.
- Include a **Dockerfile** and **docker-compose.yml** for reproducibility.

Background

You work for a logistics company that specializes in shipping goods using vessels. The company needs to track:

1. **Cargoes** – goods being transported.
2. **Vessels** – ships responsible for transporting cargo.
3. **Tracking** – movements and locations of cargo over time.

Requirements

Clients should be able to:

1. **Manage cargoes** – Create, retrieve, and update cargo status (*pending*, *in_transit*, *delivered*).
2. **Retrieve vessel details** – Name, capacity, current location.
3. **Access tracking history** – Complete movement history for each cargo.

Data Model

- Use **PostgreSQL** (preferred) or **SQLite** for simplicity.
- Define relationships between cargoes, vessels, and tracking.
- Ensure **data integrity, indexing, and normalization**.
- Implement DB migrations (you may use go-migrate, GORM migrations, or raw SQL).

Tasks

Part 1: API Development

- Build a **RESTful API**.
- Endpoints should support CRUD operations as per requirements.
- Implement proper **input validation** and consistent JSON responses.

Part 2: Database & Persistence

- Implement repository/data access layer.
- Cargo ↔️ Vessel relationship: a cargo belongs to a vessel.
- Cargo ↔️ Tracking relationship: a cargo has multiple tracking entries.
- Ensure **auditability**: cargo movements must be fully logged, not overwritten.

Part 3: Testing

- Write **unit tests** (services, handlers with mocks).
- Write **integration tests** (repositories against test DB).
- Use docker-compose to spin up a test environment (e.g., Go API + Postgres).
- Aim for **80%+ coverage**.

Part 4: Dockerization

- Provide a **Dockerfile** to build and run the API container.
- Provide a **docker-compose.yml** to run the full stack:
  - API service (Go)
  - Database (Postgres)
- The API should be accessible at *http://localhost:8080*.
- Provide a make *docker-run* command to bring up the system.

Bonus Part: Asynchronous Event Processing

Enhance your solution with **event-driven behavior**:

1. **Event Producer**
   - When cargo status changes, emit an event (*cargo_id, old_status, new_status, timestamp*).
2. **Queue Implementation**
   - RabbitMQ/Kafka (using the provided *docker-compose*).
3. **Event Consumer**
   - Background worker listens for events.
   - Logs and stores them in *cargo_events* table.

Deliverables

- A working **Golang backend API**.
- A **Dockerized setup** (Dockerfile + docker-compose.yml).
- **Tests** (unit + integration).
- **README.md** with:
  - Setup instructions (local & Docker).
  - API documentation (endpoints, request/response examples).
  - Explanation of design decisions.

Evaluation Criteria

1. **Correctness & Completeness** – All functionalities work.
2. **API Design** – REST best practices, clarity, consistency.
3. **Code Quality** – Clean architecture, idiomatic Go, modular design.
4. **Database Modeling** – Proper relationships, indexes, migrations.
5. **Error Handling & Logging** – Meaningful, structured logs and clear errors.
6. **Testing** – Good coverage, isolation, and integration with DB.
7. **Docker Expertise** – Reproducible setup with Dockerfile & docker-compose.
8. **Bonus** – Event-driven design implementation.

Submission Guidelines

- Submit as a **zip file** or GitHub repo link.
- The project should compile and run.